安全なMCPレジストリのための検証フレームワークの提案と実証評価

浦川 太良[1,a)]　中村 徹[1,b)]

概要：Model Context Protocol（MCP）は LLM エージェントと外部サービスの連携基盤として急速に普及しているが，リモート MCP サーバの不透明性に伴う新たなセキュリティリスクが懸念されている．本研究ではまず，脅威モデルとガバナンスモデルの分析に基づき，レジストリが公開前に検証すべき 3 つのセキュリティ要件（Identity, Transparency, Provenance）を策定した．次に，実際のレジストリから抽出した 231 件のサーバをこの要件で分析し，CI 運用率は約半数に達するものの Provenance 要件の完全準拠は皆無であり，不足しているのは自動化ではなく署名であることを明らかにした．さらに，ソースコード非開示のままアテステーションを発行および検証する機構を実装し，生成に約 35 秒，検証に約 1 秒という実用的な性能と，改竄されたワークフローによる署名を検知・拒否できることを実証した．また，依存パッケージの脆弱性発覚に伴い適格性が数日で失われる事例を報告し，失効機構などの継続的なガバナンスの必要性を論じる．

キーワード：LLM，Model Context Protocol，MCP レジストリ，サプライチェーンセキュリティ，アテステーション

A Verification Framework for Secure MCP Registries: Proposal and Empirical Evaluation

Taira Urakawa[1,a)]　Toru Nakamura[1,b)]

Abstract: The Model Context Protocol (MCP) has rapidly become a standard interface connecting large language model (LLM) agents to external services, yet concerns persist over the opacity of remote MCP servers and novel attack vectors. We first formulate three security requirements (Identity, Transparency, and Provenance) that registries must verify prior to publication, based on an MCP threat model and a governance analysis of comparable platforms. Second, we analyzed 231 servers drawn from two real-world registries against these requirements: while roughly half of them run CI pipelines, none satisfy the Provenance requirement—what is missing is not automation but signing. Finally, we implemented a verification mechanism using GitHub Actions and Sigstore that attests to a server's security posture without disclosing its source code, achieving practical performance (about 35 seconds to generate and 1 second to verify an attestation) and detecting signatures produced by a tampered workflow. We further report a case in which a server's attestation eligibility was lost within days of a dependency vulnerability disclosure, underscoring the need for continuous governance such as a revocation mechanism.

Keywords: LLM, Model Context Protocol, MCP Registry, Supply Chain Security, Attestation

1. はじめに

2024 年に Anthropic が公開した Model Context Protocol（MCP）[1] は，LLM エージェントと外部データソー

[1] 東京都市大学大学院 総合理工学研究科 情報専攻
Division of Informatics, Graduate School of Integrative Science and Engineering, Tokyo City University
[a)] g2681409@tcu.ac.jp
[b)] tonakamu@tcu.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

スおよびサービスを接続するデファクトスタンダードとして急速に普及している．これにより，AI エージェントは用途に特化したアダプタを個別に開発することなく，リアルタイムな情報取得や外部操作を実行できる．しかし，この接続性の拡大は，新たなセキュリティ上の懸念をもたらしている．特に，外部インフラストラクチャ上で稼働するリモート MCP サーバは，エンドユーザから内部の処理ロジックが完全に見えないブラックボックスとして振る舞う．サーバを検索し集約する MCP レジストリは，MCP エコシステムの信頼の起点となるべき存在であるが，登録されたサーバの出所や安全性を公開前に検証し保証するメカニズムは発展途上である．

本研究は，MCP エコシステム全体の健全な発展に向けて，レジストリ層がどのようなセキュリティ要件を策定し，それをどのように検証すべきかという問いに取り組む．我々はまず，古典的な Web アプリケーションの脆弱性と MCP 特有の LLM への攻撃ベクタを組み合わせた脅威モデルを整理し，MCP レジストリに移転可能なガバナンス統制を特定するため，アプリストアのクラスタリング手法を用いてそのプラットフォーム特性を分析した．その結果を踏まえ，拡張機能ストア（Chrome Web Store 等）の過去の失敗事例を参考に，レジストリが事前検証すべき 3 つの要件（**Identity, Transparency, Provenance**）を策定した．さらに，この要件を満たす検証フレームワークの一例として，ソースコード非開示のまま CI/CD と Sigstore を用いてアテステーションを検証する機構を実装した．具体的には，以下の 3 点を貢献として報告する．

- レジストリが公開前に検証すべき 3 つのセキュリティ要件（Identity, Transparency, Provenance）を，脅威モデルと類似プラットフォームのガバナンス分析に基づき策定した．
- 実在する 2 つのレジストリから抽出した 231 件のサーバを策定した要件で採点し，Provenance 要件の完全準拠が全くないことを実証した．
- 要件を満たす検証手法の例としてアテステーション機構を実装し，実用的な性能に加え，実世界の運用における失敗モード（アテステーション適格性の時間減衰や実行時非再現性）を明らかにした．

本論文の構成は以下のとおりである．2 章で MCP の技術背景と脅威を整理し，3 章で関連研究との差分を述べる．4 章でレジストリが検証すべき 3 要件を定義し，5 章と 6 章で実験の設計と結果を示す．7 章で考察を行い，8 章でまとめる．

2. 技術背景

2.1 Model Context Protocol

MCP のアーキテクチャは，**図 1** に示すようにホスト，クライアント，サーバの 3 つの主要コンポーネントから

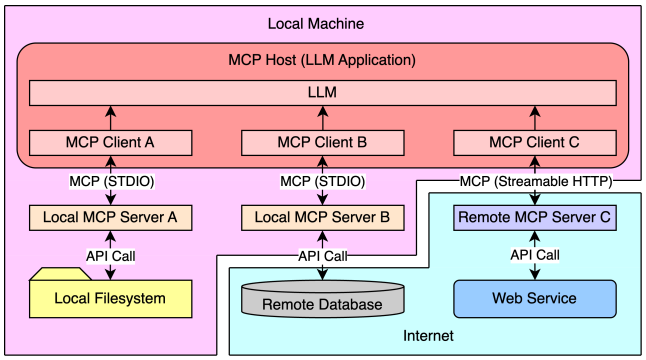


図 1 MCP の構成要素
Fig. 1 The Architecture Overview of the MCP Components.

構成され，それぞれが連携してコンテキスト検索を処理する．ユーザの入力を受け取った MCP ホストは，LLM への橋渡しと MCP クライアントの生成を担う．生成された MCP クライアントは，対応する MCP サーバに要求を送信する．MCP サーバは，MCP の仕様を実装した軽量なプログラムとして，ローカルデータや外部サービスへの実際のアクセスを提供する．サーバが接続されると，ユーザ制御の Prompts，アプリケーション制御の Resources，モデル制御の Tools という 3 つのプリミティブを公開する．このうち Tools は，LLM が情報取得や外部操作のために呼び出す実行可能関数であり，入出力スキーマによって記述される．実行環境の違いは，サーバのセキュリティプロファイルを大きく変える．ローカル MCP サーバは標準入出力を介してユーザの端末上で実行され，npm[2] や PyPI[3] などの公開パッケージレジストリを通じて配布されるため，ソースコードを検査できる．一方，リモート MCP サーバは Streamable HTTP を介して外部インフラストラクチャ上で実行される．その内部ロジックはユーザから完全に不可視であり，ブラックボックスとして振る舞う．

2.2 MCP レジストリ

MCP レジストリは，MCP サーバに関するメタデータや機能を一元的に集約し，検索の拠点として機能するサービスである．実行コードそのものは保持せず，メタデータの提供に限られる [4]．代表例として，公式の MCP レジストリ [5] と，コミュニティ主導の Smithery[6] がある．両者ともディスカバリ機能は提供しているが，登録されたリモート MCP サーバの動作の安全性を保証する仕組みは備えていない．

2.3 MCP サーバのセキュリティ脅威

MCP のエコシステムは，従来の Web アプリケーションが抱えるセキュリティ脅威（SQL インジェクションや SSRF など）を同様に抱えている [7]．実際に，公式の SQLite サーバ参照実装において SQL インジェクションが発見された事例 [8] や，公開パッケージを通じたサプライチェーン攻

© 2026 Information Processing Society of Japan

撃（例：npm の Shai-Hulud[9]）のリスクが存在する．

これに加え，MCP に固有の脅威が存在する．その多くは，LLM がツールの記述文を読んで呼び出す対象と引数を決めるという構造に起因する．攻撃者はツール説明に指示を埋め込むことで，コードを改変せずに LLM の挙動を操作できる（ツールポイズニング [10], [11], [12]）．この操作は，説明文が無害に見えたまま実装側でデータを外部へ送出する形にもなりうる．さらに，ツールが実行時に取得した外部データに指示が埋め込まれていれば，記述文自体に問題がなくとも同種の操作が成立する（RADE[13]）．前者はサーバのソースコードに痕跡が残るが，後者は登録時点のコードには存在しない．これらとは別に，登録時に検証したコードと稼働中のサーバが実行するコードが一致する保証もない．リモート MCP サーバの内部は不可視であり，検証済みの成果物と稼働物を結びつける手段が存在しないためである．本稿ではこの乖離を**デプロイメントギャップ**と呼ぶ（TTP 分類 [14] の `Drift from Upstream` に対応する）．

こうした攻撃手法は，コミュニティによる TTP 分類では 12 カテゴリ 71 技術として [14]，Guo らは 31 種の攻撃を 4 分類として [15] 整理している．また，Li と Gao は 67,057 件のサーバを対象とした大規模な実測において，リンクの健全性や認証情報の漏洩，アカウント乗っ取りといったレジストリ層の脅威を報告している [16]．個別の技術の詳細は文献 [14], [15], [16] に譲る．登録時点でレジストリが検証しうる範囲の脅威について，TTP 分類の 12 カテゴリに沿って本フレームワークの対象範囲を整理したものを付録の**表 A·1** に示す．

本研究が登録時点で扱うのはサプライチェーンと依存関係，コードおよびコマンドの注入，認証情報の漏洩，監査証跡の各カテゴリであり，プロンプトインジェクションと文脈操作は対象外とする．すなわち本フレームワークは攻撃面全体を覆うものではない．実行時層の脅威は原理的に登録時点では観測できず（第 6.4 節で観測境界を実測する），実行時防御との層の分業を前提とする．

2.4 MCP レジストリのプラットフォーム特性

MCP レジストリがどのようなプラットフォームに位置づけられるかを明らかにするため，Zhu ら [17] によるアプリストアのクラスタリング手法を適用した．Zhu らのデータセットが用いる 41 の二値特徴量（CI/CD 連携や Pre-publication checks の要否等）により Smithery を符号化し，PCA および k-means 法を用いた分類を行った（詳細は付録 A.6 節）．結果として，Smithery は単なるパッケージマネージャではなく，コミュニティ主導の非特化型拡張ストア（Chrome Web Store[18] や Firefox Add-ons と同じクラスタ）に分類された．この事実は，MCP レジストリに対して，拡張ストア特有のガバナンスモデル（事前検証と継続的監視）を適用すべきであることを示唆している．ただし本分類は，移転可能なガバナンス統制を特定するための道具として用いるものであり，リモート MCP サーバがブラウザ拡張と等価であると主張するものではない．配布とガバナンスの特性は拡張ストアに近い一方で，配布される対象は実行時に外部と通信する動的なサービスであり（第 6.1 節），両者の性質を併せ持つ．この二重性ゆえに，拡張ストア由来の事前検証だけでは不十分であり，公開後の継続的な検証が要求される．

2.5 類似プラットフォームにおけるセキュリティ対策

Smithery と同じクラスタに属する Chrome Web Store では，WebEval[19] と呼ばれる静的および動的解析と，専門家判断を組み合わせた事前検証システムが運用されている．しかし，一度審査を通過した拡張機能が後続の更新において悪性化する事例が，大規模な調査により多数報告されている [20]．更新は同一の認証済み開発者アカウントから提出されるため，発行者の身元が検証されていることは成果物の安全性を意味しない．事前検証のみでは公開後の安全性を保証できず，リモート MCP サーバの検証においても継続的な検証が必要となる．

3. 関連研究

Mas ら [21] は，MCP レジストリのサプライチェーン汚染を防ぐため，3 層のアーキテクチャを提案した．彼らの手法は，RFC 8615（/.well-known/mcp.json）による分散ディスカバリ，Sigstore を用いたビルド成果物の Provenance，そして RFC 8785（JCS）に基づくツールリスト変更の署名検証によって構成される．本研究の要件フレームワークは彼らの設計と目的を共有しているが，以下の 3 点で決定的に異なる．

第一に，評価設定である．Mas らは合成環境におけるプロトタイプを用いたシミュレーション評価を行っている．本研究は，彼ら自身が将来の課題として挙げた実世界のデプロイメントに焦点を当て，実在する 231 台のサーバを採点し，さらに 10 台のサーバをフォークして実 CI 環境で評価した．第二に，脅威モデルである．彼らはパイプラインを管理する正規の開発者を信頼側に置いている．本研究は，悪意ある開発者がスキャンゲートを弱めた改竄ワークフローを作成するシナリオを想定し，彼らの設計上の反例を実証する．第三に，証明する命題である．彼らはビルド成果物のハッシュを対象とし，成果物がそのリポジトリの CI から生成されたこと，すなわち出所を証明する．本研究はセキュリティスキャン結果と SBOM を対象とし，そのコードがレジストリの規定する検査を通過したこと，すなわちポリシーの遵守を証明する．出所の証明はポリシーの遵守を含意しない．この差は第二の相違点と結びついている．出所のみを証明する設計では，開発者が自ら定義し

© 2026 Information Processing Society of Japan

たワークフローによる署名も要件を満たしてしまうためである．

なお，Guo ら [15] は MCP に対する 31 種の攻撃を 4 分類に体系化し，攻撃ツールキット MCPXKIT として実装した上で各攻撃の有効性を定量化したが，同分類の大半はサーバ統合後のツール記述と LLM の相互作用を対象としており，登録・配布時点での検証は扱っていない．Wang ら [22] は MCP の防御手法を体系的に調査し，サーバ側の事前走査と実行時監視に大別した上で，ゼロトラスト型のレジストリ設計を前者の一類型として位置づけている．

レジストリ層に踏み込んだ議論として，Errico ら [23] は，現状のレジストリが登録要件としてリポジトリまたはドメインの所有証明のみを課し，コードレビューやセキュリティ監査を求めていないことを指摘し，公式レジストリへの掲載が検証を経た証にならないにもかかわらず利用者が検証済みと誤解する点を「誤った安心感」と呼んで問題視している．そして同研究は，再現可能ビルドとコード署名の組み合わせや透明性ログの活用を挙げつつ，「検証可能なレジストリ」の設計を未解決の研究課題として明示している．本研究はこの課題に対し，レジストリが登録前に検証すべき要件を定め，実際の CI/CD 環境上で署名付きアテステーションを用いた検証機構を実装し，その充足実態と実用性を実測によって評価する．

4. セキュリティ要件とフレームワーク

前節までの分析から，レジストリが登録を受理する前に答えられなければならない問いは 3 つに整理できる．すなわち，誰が公開したのか（**Identity**），何ができると宣言しているのか（**Transparency**），そしてその実装が主張どおりの過程を経たのか（**Provenance**）である．第 2.3 節で整理した脅威のうち登録時点で観測可能なものは，いずれかの問いに帰着する（詳細は付録 A.1 節）．また，拡張ストアにおいて事前審査を通過した後に乗っ取りが生じた事例（第 2.5 節）は，これらの検証が登録時点で完結せず，公開後も継続する必要があることを示している．

4.1 セキュリティ要件

本節では，MCP レジストリがリモート MCP サーバに信頼を与える前に検証すべきセキュリティ要件である Identity, Transparency, Provenance について詳細に述べる．

4.1.1 Identity

誰がサーバを公開したのか，そして主張するドメインや名前空間を実際に管理しているかを検証する要件である．具体的には，OpenID Connect（OIDC）を用いたログインによる開発者認証と，DNS 等を通じたドメイン所有権の証明が含まれる．ただし，Identity はセキュリティそのものを保証しない．身元確認は責任の追及を可能にするが，その開発者が公開したコードが安全であることの証明にはならない．

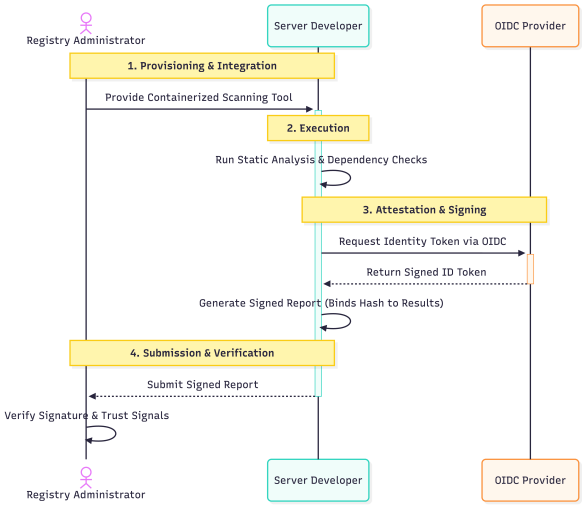


図 2 ソースコード非開示のアテステーション検証フロー
Fig. 2 The Workflow of Attestation-Based Auditing without Source Submission.

4.1.2 Transparency

そのサーバが何を実行できると宣言しているかを明示する要件である．リモート MCP サーバの内部を直接検査できない以上，ユーザやクライアントアプリケーションは接続前にリスク評価を行うための情報が必要となる．この要件は，ツールの権限開示，ケイパビリティマニフェストの提示，および SBOM の提供によって満たされる．

4.1.3 Provenance

公開されたサーバのコードが，主張どおりの CI/CD パイプラインを通過したかを確認する要件である．本研究における Provenance は，ビルドの出所を証明する従来の定義を拡張し，セキュリティ解析の結果そのものを署名の対象に含める．レジストリが規定する CI/CD ワークフロー（たとえば静的解析や依存関係スキャン）を実行し，その結果を Sigstore 等を用いた署名付きアテステーションとして出力する．このとき，規定した検査に合格しない限りアテステーションを発行しないという合否判定条件を設ける．本稿ではこれを以下**ゲート**と呼ぶ．さらに，検証時には OIDC のクレームに含まれる `job_workflow_ref` を確認し，規定のワークフローが改竄されずに実行されたことを保証する．この機構により，ブラックボックスであるリモート MCP サーバのソースコードを開示させることなく，レジストリはサーバのセキュリティ上の健全性を検証できる．

4.2 提案フレームワークの概念とワークフロー

本フレームワークは，ソースコードの完全開示というプロプライエタリな開発者にとっての障壁を取り除きつつ，セキュリティ上の健全性を証明可能な形でレジスト

© 2026 Information Processing Society of Japan

リに提供する．具体的なワークフローは，**図 2** に示す 4 つのフェーズで構成される．第 1 フェーズ（Provisioning & Integration）では，レジストリ運用者が必須のセキュリティチェックと出力形式を定義した公式スキャンプロファイルをコンテナ化された検査ツールとして配布し，開発者がこれを自らの CI/CD パイプラインに統合する．第 2 フェーズ（Execution）では，CI/CD 上で SAST や SCA による静的解析と依存関係検査が実行される．第 3 フェーズ（Attestation & Signing）では，検査に合格した場合に OIDC を介して身元トークンを取得し，コードのハッシュと解析結果をまとめて署名したレポートを生成する．第 4 フェーズ（Submission & Verification）では，開発者がこのレポートをレジストリに提出し，レジストリが署名と信頼シグナルを検証して信頼マーク（Verified バッジ等）を付与する．

4.3 信頼シグナルとしての役割

このワークフローから出力されるアテステーションは，機械可読な**信頼シグナル（Trust Signals）**として機能する．これには，OIDC に裏付けられた発行者身元（Identity 要件の証拠），ケイパビリティの宣言（Transparency 要件の証拠），および SBOM や脆弱性スキャン結果（Provenance 要件の証拠）が含まれる．これらを組み合わせることで，レジストリおよびクライアントはソースコードを検査せずにリスクベースの接続判断を行うことができる．

5. 実験設計

本研究では，要件フレームワークの妥当性と提案機構の実用性を検証するため，5 つの実験を設計した．

(1) **Exp 1 (Registry Gap Analysis)**：実レジストリのサーバが 3 要件をどの程度満たすかを定量化する．
(2) **Exp 2 (CI/CD Attestation PoC)**：ソースコード非開示でアテステーションを発行・検証する機構の構築可能性とコストを計測する．
(3) **Exp 3 (Scanning-Profile Integrity)**：パイプラインを管理する開発者が，ゲートを弱めた改竄プロファイルから有効な署名を作成できるかを検証する．
(4) **Exp 4 (agent-scan Comparison)**：プロトコル層を監視する agent-scan と本手法の観測境界を比較し，補完性を実証する．
(5) **Exp 5 (Deployment-Gap Mitigation)**：アテステーション発行済みのコードが実際に実行されているか（実行時整合性）を検証する．

6. 実験結果

6.1 実レジストリにおける要件充足率（Exp 1）

本研究の採点は 3 要件に対応する 8 項目の充足度評価であり，各サーバについて tarball の取得とリポジトリ構造の検査を要する．リンクの生死判定のみを行う Li と Gao[16] の大規模測定とは 1 サーバあたりの評価コストが異なるため，シード固定の無作為抽出を採用した．公式 MCP レジストリと Smithery のそれぞれから各 200 件（計 400 件）を抽出し，GitHub リポジトリの解決に成功した 231 件（57.8%）を評価対象とした．解決率は公式レジストリが 73.5%（147 件），Smithery が 42.0%（84 件）である．なお Smithery の公開リストは 5 ページが上限で重複を含み，重複除去後のユニークは約 253 件であった．結果を**表 1** に示す．Identity 要件や Transparency 要件については部分準拠以上のサーバが多数存在する一方で，Provenance 要件の完全準拠は 0.0%であった．CI/CD パイプラインを運用しているサーバ（P-1）が 51.9%に達する一方で，公開された署名付きアテステーション（P-2）を持つサーバは皆無だった．この実測値は，レジストリの Provenance 検証において不足しているのは自動化の仕組みではなく，署名そのものであることを示している．全体の 92.6%は認証やネットワークアクセスを伴うサービス型のサーバであった．この結果は，第 2.4 節で述べた二重性を実測で裏付けるものである．すなわちレジストリは拡張ストアに近いガバナンス特性を持つ一方で，配布される対象の大半は実行時に認証情報や外部通信を扱う動的なサービスであり，公開時点の静的な審査だけでは信頼を担保できない．

表 1 レジストリにおける 3 要件の充足率 (N=231)．全項目の充足を完全準拠，一部の充足を部分準拠，充足なしを非準拠とした (付録 A.2 節)

Table 1 Compliance Rates of the Three Requirements (N=231).

要件	完全準拠	部分準拠	非準拠
Identity	25.1%	60.6%	14.3%
Transparency	8.7%	67.1%	24.2%
Provenance	0.0%	51.9%	48.1%

この実態は，現在のレジストリが自動的な強制力よりもコミュニティの信頼に依存していることの反映である．Official Registry は OSS のプルリクエストレビューに依存し登録後の動作変更を追跡できず，Smithery は agent-scan[24] を導入するものの稼働中のサーバが `tools/list` で返すメタデータのみを検査する．MCP Market[25] や MCP.so[26] に至っては，深い検証を伴わない一覧の提供にとどまる [23]．すなわちメタデータ開示のみに頼る現状のアプローチでは，リモート MCP サーバの不透明性は解決されない．

6.2 アテステーション機構の実装と計測（Exp 2）

実在する 10 件の OSS サーバをフォークし，実 CI 環境上で Semgrep および Trivy によるゲートを含むアテステーション生成機構を稼働させた．仕様どおりの厳格ゲート（1

© 2026 Information Processing Society of Japan

件でも検出があれば不合格）では 0/10 が通過し，測定が不能となった．ただしこの測定には，注入した検査ワークフロー自身に対する検出（全リポジトリで一律 5 件，いずれもアクションのタグ非固定参照の指摘）が含まれている．これを除いた対象サーバのコード由来の検出は 0 件から 59 件の範囲であり，検出 0 件のサーバも 1 件存在した．すなわち厳格ゲートが成立しない原因は対象コードの品質のみではなく，検査ワークフロー自体が同じ厳格な基準を満たさないことにもある．そこで，深刻度が ERROR，HIGH，または CRITICAL の項目のみをブロックするようにゲートを調整した結果，3/10 のサーバが通過した（**表 2**）．同表の SAST 列は上記の一律 5 件を除外した対象コード由来の検出数を，その括弧内はうちゲート対象となる ERROR 深刻度の件数を，SCA 列は Trivy が検出した HIGH 以上の依存脆弱性件数を示す．この通過率は実世界のコード品質を反映した数値であり，機構自体の限界ではない．

さらに対象コード由来の 126 件について，真陽性と偽陽性の手動判定（トリアージ）を実施した．誤検知は 4 件（3.2%）にとどまり，いずれも非暗号用途のハッシュ利用または変数名に基づくヒューリスティックであった．悪用可能な欠陥は 3 件（2.4%）であり，残る 119 件（94.4%）はタグ非固定参照やコンテナの root 実行など，指摘自体は正しいが直接の悪用経路がないものであった．この分布は厳格ゲートが実用的でない理由を説明する．検出の大半は悪用可能な脆弱性ではなく，悪用経路のない指摘であり，深刻度による選別も悪用可能性と一致しない（ERROR 判定 9 件のうち悪用可能は 2 件）．レジストリはゲート条件を，深刻度ではなく悪用可能性に基づいて設計する必要がある．

ゲートを通過した 3 件（N=3）におけるアテステーション生成時間の平均は 34.7±3.1 秒であった．一方，署名検証，発行者信頼，ワークフロー妥当性，鮮度，およびダイジェストの一致を確認する検証コストは，平均約 1 秒（1.162 秒）であった．これはソースコード非開示のまま検証を行う上で，実用的な性能である．

6.3 改竄ワークフローの検証（Exp 3）

Exp 2 の機構において，スキャンゲートを除去した改竄ワークフローを意図的に実行し，署名付きアテステーションを発行させた．署名は完全に有効なものであり，発行者は信頼された GitHub Actions の OIDC で，ダイジェストも SBOM と一致する．しかし，OIDC クレームに含まれる `job_workflow_ref` がレジストリ側で信頼する特定のファイルパス（例：`.github/workflows/mcp-registry-attest.yml`）に合致しないという一点のみによって，このアテステーションは検証器に拒否された．なお，あるリポジトリのアテステーションを別のリポジトリの成果物に付け替える攻撃についても，ダイジェスト照合により拒否されることを確認した．Provenance はビルドの出所を証明するが，ポリシーの遵守までは証明しない．本機構により，パイプラインを握る開発者であっても，ゲートを弱めたプロファイルからレジストリが受理する署名を作成できないことが実証された．

6.4 既存スキャナとの補完性（Exp 4）

プロトコル層のツール表面を監視する既存のセキュリティツール（agent-scan）と，本手法（ソース層の検査）のカバレッジを比較した．意図的にデータを窃取する動作を埋め込んだ PoC サーバに対し，双方の検査を適用した．agent-scan はプロンプトインジェクションや不可視 Unicode など，ツール説明文の表面に現れる脅威を捕捉した．しかし，ソース層における攻撃のマーカー（ハードコードされた認証情報や隠れた外部への送信処理）の表面への漏出は 0 件であった．結果として，本手法の Semgrep が AWS キーや隠れた送信処理を，Trivy が 12 件の CVE をソース層で検出した．Guo ら [15] の定量評価において最も有効性が高い攻撃（SQL Injection や API Theft 等）がプロトコル表面の走査では見えない層にあることからも，本手法の有用性が裏付けられる．プロトコル層の監視とソース層の Provenance 検証は，異なる脅威をカバーする補完的な関係にある．ただし，ソース層の検査がプロトコル層の記述を包含して検証できるのは，デプロイされた稼働コードと検査対象のコードが一致している場合のみである．登録時点での検査結果が稼働時点でも維持されていることを担保するためには，次節の検証が必須となる．

6.5 実行時整合性の検証（Exp 5）

アテステーションの対象となったコミットをチェックアウトして起動し，その `tools/list` のハッシュがアテステーション内の値と一致するかを確認した．倫理上の制約から第三者の稼働サーバへは接続せず，自身が管理するフォークを標準入出力経由で起動して稼働環境の代替とした．コミュニティの脅威分類 [14] における「Drift from Upstream」への対策として，SBOM 内にツールの正規化ハッシュ（名前順にソートしたツールスキーマの SHA-256 値）を付与し，実行中のサーバが返すツールリストのハッシュと突き合わせる検証チェーン（付録 A.5 節）を構築した．対象とした 10 サーバすべてにおいて，CI 上で取得したハッシュと実行時のハッシュの整合性が確認できた（10/10）．ただし，Sigstore バンドルの検証まで成立したのは 2 件であった．これは Exp 2 でゲートを通過した 3 件と一致しないが，方法論の差ではなく，計測期間中に依存パッケージの脆弱性が公開され 1 件が資格を失ったことによる（第 7.2 節）．残る 8 サーバについては，ハッシュの一致は確認できるが署名の対象に含まれていない状態であり，完全な検証チェーンが成立するのはゲートを通過した

© 2026 Information Processing Society of Japan

表 2　アテステーション生成の PoC 評価 (N=10)
Table 2　Proof-of-Concept Evaluation for Attestation Generation.

サーバ	SAST (ゲート対象)	SCA	生成 (s)	SBOM (KB)	ゲート判定
upstash/context7	28 (3)	5	—	104.4	拒否
punitarani/fli	59 (2)	18	—	73.3	拒否
zwldarren/akshare-one-mcp	11 (1)	14	—	53.6	拒否
cyanheads/pubmed-mcp-server	1 (0)	0	32.0	86.4	受理
cyanheads/clinicaltrialsgov-mcp-server	4 (0)	0	38.0	67.5	受理
pinkpixel-dev/mindbridge-mcp	1 (1)	0	—	0.9	拒否
mikechao/metmuseum-mcp	8 (1)	7	—	62.2	拒否
linxule/lotus-wisdom-mcp	7 (0)	12	—	183.3	拒否
ByteAsk/ByteAsk-Embedded-MCP	0 (0)	0	34.0	23.6	受理
adamamer20/paper-search-mcp-openai	7 (1)	19	—	22.3	拒否

サーバに限られる．一方，アテステーション発行後にツールの説明文を変更する改竄シナリオでは，直ちに不一致が検出された．

7. 考察

7.1 既存検証アプローチとの比較

ソースコードの完全提出を要求しレジストリ側でスキャンを実行するモデル（例えば Li と Gao による MCPInspect[16]）は最大の透明性を得られるが，レジストリ側の検証コストが膨大になり，プロプライエタリなソフトウェアの公開障壁となる．他方，公開後にコミュニティの報告を待つ事後報告モデルは受動的であり，事前検証としては不十分である．本手法はソースコードを秘匿したままセキュリティ上の健全性を証明できる点で，これらの中間に位置する実用的な解である．

7.2 Identity の有効性と Provenance の適格性の時間減衰

レジストリが一度付与した信頼は恒久的なものではない．第一に，登録時に確認した発行者の身元は，時間の経過とともに実態と乖離しうる．Li と Gao[16] の大規模調査によれば，アカウントの削除や改名により数百件規模で既存サーバの乗っ取りが可能であった．第二に，本研究の Exp 5 が示すように，Provenance の適格性も脆弱性データベースの進行とともに数日で失効する．7 月 17 日にゲートを通過したサーバが，依存関係（mcp 1.28.0）を変更していないにもかかわらず 3 日後の 7 月 20 日に不合格となった．原因は MCP SDK 自体の CVE-2026-59950（CVSS 8.1，Host/Origin 検証の欠如）だが，同脆弱性は最初のスキャンの 2 日前，7 月 15 日に公開されていた．変化したのはコードでも依存でもなく，スキャナのデータベースの状態のみである．すなわちアテステーションが証明するのは成果物の安全性ではなく，スキャン時点でスキャナが把握していた範囲にすぎない．これらの事実は，Identity も Provenance も一度検証すれば永続する保証ではないという統一的な課題を示している．失効の仕組み自体はエージェントレジストリの設計論において既に提示されている（NANDA における VC-Status を用いた即時失効など [4]）．一方，公開されている仕様および文献の範囲では，MCP レジストリに同種の失効機構は確認できない．実測においても公開された署名付きアテステーションは 0.0%であり，そもそも失効の対象となる証明自体が運用されていない．本研究の結果は，発行後の再評価を含む継続的なガバナンスが必要であることを示している．

7.3 デプロイメントギャップと実行時再現性

アテステーション検証の原理的な限界として，CI/CD で検証されたコードと，実際に外部インフラで稼働しているリモート MCP サーバのコードが異なるデプロイメントギャップが存在する．Exp 5 では実行時完全性の検証を試みたが，Python の docstring からツール説明を生成するサーバにおいて，Python 3.13 でのコンパイル時のインデント仕様変更に起因してツールのハッシュ値が一致しない事象が発生した（検査側を`--python 3.12` に固定して再現させた）．この実行時非決定性は，再現可能な実行環境の構築が検証の成立条件であることを示している．なお本実験は機構の成立性を管理下の環境で示したものであり，実運用ではレジストリが稼働サーバの `tools/list` を当該トランスポート越しに取得して同じ照合を行う必要がある．このギャップに対する技術的選択肢は到達点が異なる．ビルドの決定性を保証する「Reproducible Builds」と成果物の保持を証明する「Provable Data Possession」[27] は，いずれも成果物が実際に稼働しているかを示さず，正規の成果物を保管したまま別プロセスに処理させる攻撃を排除できない．実行時完全性に到達しうるのは TEE を信頼の基点とする「Remote Attestation」のみであり，残る 2 つは攻撃コストを引き上げる緩和策にとどまる．ただし TEE もロード時の測定値を保証する技術であり，ハードウェアベンダへの信頼と運用者の対応環境を前提とする．

© 2026 Information Processing Society of Japan

7.4 スキャンの限界と分散ガバナンス

本研究にはいくつかの限界がある．第一に，自動化されたスキャンには誤検知と見逃しのトレードオフが常に存在し，静的解析だけでは実行時のみ発現する高度な難読化を捕捉できない可能性がある．本研究のトリアージは単一の評価者によるものであり，複数評価者による判定一致率の検証は行っていない．また，Transparency 要件におけるケイパビリティ宣言と実装の網羅性の検査については，要件として定義したものの，本稿の実験機構には実装されておらず未実証である．第二に，本フレームワークは中央集権的なアーキテクチャであり，システムの信頼は最終的にレジストリ運用者に帰着するため，運用者が侵害された場合は単一障害点となる．第三に，生成コストの計測点が N=3 と少ないため，より大規模な実証が必要である．これらの課題を解決するためには，複数レジストリ間でのトラスト共有や台帳ベースのアテステーション記録など，分散ガバナンスモデルへの移行が今後の研究課題となる．

8. おわりに

本研究は，MCP レジストリにおけるサーバの安全性を担保するため，脅威モデルとガバナンスモデルの分析に基づく Identity，Transparency，Provenance の 3 要件を策定した．実在する 231 サーバの実測により，Provenance 要件の欠如という現状を定量化し，要件を満たす検証手法の一例としてアテステーション機構を実装した．この実証評価により，実用的な性能や改竄耐性に加え，アテステーション適格性の時間減衰といった運用上の課題を明らかにした．今後の課題として，失効機構の設計，分散化アーキテクチャの検討，および計測点数の拡大が挙げられる．

参考文献

[1] Anthropic: Introduction to the Model Context Protocol, `https://www.anthropic.com/news/model-context-protocol` (2024).

[2] npm: npm — Home, `https://www.npmjs.com/` (2026).

[3] Python Software Foundation: PyPI · The Python Package Index, `https://pypi.org/` (2026).

[4] Singh, A., et al.: Evolution of AI Agent Registry Solutions: Centralized, Enterprise, and Distributed Approaches, Proc. IEEE 7th Int. Conf. on Cognitive Machine Intelligence (CogMI), pp.507–515 (2025).

[5] Model Context Protocol: Official MCP Registry, `https://github.com/modelcontextprotocol/registry` (2025).

[6] Smithery: The registry for MCP servers, `https://smithery.ai/` (2025).

[7] Equixly: MCP Servers: The New Security Nightmare, `https://equixly.com/blog/2025/03/29/mcp-server-new-security-nightmare/` (2025).

[8] Trend Micro: Why a Classic MCP Server Vulnerability Can Undermine Your Entire AI Agent, `https://www.trendmicro.com/en_us/research/25/f/why-a-classic-mcp-server-vulnerability-can-undermine-your-entire-ai-agent.html` (2025).

[9] CISA: Widespread Supply Chain Compromise Impacting npm Ecosystem, `https://www.cisa.gov/news-events/alerts/2025/09/23/widespread-supply-chain-compromise-impacting-npm-ecosystem` (2025).

[10] Hasan, M. M., et al.: Model context protocol (mcp) at first glance: Studying the security and maintainability of mcp servers, arXiv:2506.13538 (2025).

[11] Hou, X., et al.: Model context protocol (mcp): Landscape, security threats, and future research directions, arXiv:2503.23278 (2025).

[12] Kumar, S., et al.: Mcp guardian: A security-first layer for safeguarding mcp-based ai system, arXiv:2504.12757 (2025).

[13] Radosevich, B. and Halloran, J.: Mcp safety audit: Llms with the model context protocol allow major security exploits, arXiv:2504.03767 (2025).

[14] Model Context Protocol Security: TTP Matrix View, `https://modelcontextprotocol-security.io/ttps-view/` (2025).

[15] Guo, Y., et al.: MCPXKIT: The Unified Toolkit for Analyzing Model Context Protocol Security, *IEEE Transactions on Dependable and Secure Computing* (2026). DOI: 10.1109/TDSC.2026.3695553.

[16] Li, X. and Gao, X.: A First Look at the Security Issues in the Model Context Protocol Ecosystem, Proc. IEEE/IFIP Int. Conf. on Dependable Systems and Networks (DSN) (2026).

[17] Zhu, W., et al.: What is an app store? The software engineering perspective, *Empirical Software Engineering*, Vol.29, No.1, p.35 (2024).

[18] Chrome for Developers: Chrome Web Store review process, `https://developer.chrome.com/docs/webstore/review-process` (2021).

[19] Jagpal, N., et al.: Trends and lessons from three years fighting malicious extensions, *24th USENIX Security Symposium*, pp.579–593 (2015).

[20] Pantelaios, N., Nikiforakis, N. and Kapravelos, A.: You've Changed: Detecting Malicious Browser Extensions through their Update Deltas, *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pp.477–491 (2020).

[21] Mas, L., Vilaplana, J., Rius, J., Spaimoc, R., and Mateo, J.: The Trustworthy Model Context Protocol (MCP) Registry: An Architectural Blueprint for Cryptographic Provenance and Runtime Integrity, *Future Internet*, Vol.18, No.5, p.243 (2026).

[22] Wang, Z., et al.: MCPGuard: Automatically Detecting Vulnerabilities in MCP Servers, arXiv:2510.23673 (2025).

[23] Errico, M., et al.: Securing the Model Context Protocol (MCP): Risks, Controls, and Governance, arXiv:2511.20920 (2025).

[24] Snyk: agent-scan (旧 mcp-scan)— Security scanner for AI agents, MCP servers and agent skills, `https://github.com/snyk/agent-scan` (2026).

[25] MCP Market, `https://www.mcp.market/` (2025).

[26] MCP.so, `https://mcp.so/` (2025).

[27] Ateniese, G., et al.: Provable Data Possession at Untrusted Stores, *Proceedings of the 14th ACM Conference on Computer and Communications Security*, pp.598–609 (2007).

© 2026 Information Processing Society of Japan

9. 研究倫理

本研究は Exp 2 および 3 において，実在するオープンソースの MCP サーバ（N=10）の GitHub リポジトリをフォークし，CI/CD 環境上でスキャン（Semgrep，Trivy）とアテステーション発行を実施した．対象はすべて MIT または Apache-2.0 ライセンスの公開プロジェクトであり，ライセンス範囲内の行為である．Exp 1 では実サーバへ一切接続せず，GitHub REST API と公開 tarball のみを参照した．Exp 3 の改竄ワークフローと Exp 4 の模擬データ窃取攻撃は，いずれも研究者の管理下にあるフォークおよび自作 PoC サーバ内に封じ込め，後者は未設定フラグと到達不能ドメイン（.invalid TLD）で無害化しており，第三者インフラへの影響は生じない．Trivy の検出結果はすべて公開済みの CVE であり新規脆弱性の開示には該当しない．また Semgrep の secrets 検出ルールは実在サーバに対して検出を示さず，認証情報に関連する検出は GitHub Actions の権限委譲設定（`secrets: inherit`）と誤検知 1 件のみであった．いずれも実際の認証情報を含まないため，責任ある開示を要する事案は生じていない．また，実在サーバに対する検出のうち悪用可能と判定した項目については，件数のみを集計として報告し，個別のサーバへの帰属や該当箇所は記載していない．これらはいずれも公開されているコードに対して標準的なルールセットで静的に再現可能な既知パターンの指摘であり，非公開の情報を含まない．

付　　録

A.1 要件フレームワークの対象範囲

本フレームワークが対象とする脅威と対象外の脅威を，コミュニティによる TTP 分類 [14] の 12 カテゴリに沿って表 A·1 に整理する．各行には対応する要件（I: Identity，T: Transparency，P: Provenance）と，その根拠となる実験を併記した．

A.2 Exp 1 の採点ルーブリック

各サーバを表 A·2 の 10 項目について 0/1 の二値で判定した．要件ごとの準拠レベルは，当該要件の全項目を充足した場合を完全準拠，1 項目以上を充足するが全部ではない場合を部分準拠，1 項目も充足しない場合を非準拠とした．判定はレジストリ API，GitHub REST API，および公開 tarball のみに基づき，MCP サーバへは接続していない．

P-2 は署名済み成果物が既に存在するかの事後確認であり，P-3 は署名の意図の確認である．両者を分けたことで，本文第 6.1 節で述べた「CI は動いているが署名がない」という非対称性を項目単位で観測できる．S-1 と S-2 はサービス型特性の判定に用い，いずれかを満たすサーバをサービス型とした．

なお実測にあたり 2 点の制約が生じた．第一に，汎用の DNS TXT レコードによるドメイン所有権の照会は不可能であったため，I-1 はリポジトリ所有者の GitHub プロフィールに登録されたドメインを代理指標とした．第二に，S-1 の判定において HEAD リクエストによる検査は，リモート MCP サーバへ接続しないという倫理上の制約により省略し，README の記述のみで判定した．

A.3 検証器の判定条件

Exp 2，3，5 で用いた検証器は，提出された Sigstore バンドルに対して以下の 5 項目をすべて検査し，1 項目でも不合格であればアテステーションを拒否する．

(1) **署名の有効性**：バンドルの署名が Fulcio 発行の証明書に対して検証できるか
(2) **発行者の信頼性**：証明書の発行者が信頼リストに含まれるか
(3) **ワークフローの妥当性**：`job_workflow_ref` がレジストリの規定するワークフローを指すか
(4) **鮮度**：アテステーションの発行時刻が許容期間内か
(5) **ダイジェストの一致**：subject のダイジェストが提出された成果物と一致するか

信頼設定は次のとおりである．発行者は `token.actions.githubusercontent.com` および `gitlab.com` を信頼し，ワークフローは末尾が `.github/workflows/mcp-registry-attest.yml` であることを要求する．第 3 項目における `job_workflow_ref` は証明書の SAN URI および Build Signer URI（OID 1.3.6.1.4.1.57264.1.9）に記録される．OID 1.3.6.1.4.1.57264.1.6 は git ref のみを保持するため，ワークフローの同定には用いられない点に注意を要する．本文第 6.3 節で報告した改竄ワークフローの拒否は，第 3 項目のみが不合格となった結果である．

A.4 Exp 2 の対象選定とトリアージ基準

Exp 2 の対象は，Exp 1 で Smithery から取得したサーバのうち，GitHub リポジトリが解決でき，スター数が 10 を超え，MIT または Apache-2.0 ライセンスであり，Python または TypeScript で実装され，README にツールの記述があるものを候補とし，スター数上位 10 件を決定的に選定した．

検出項目のトリアージは以下の 3 分類で行った．

誤検知　検出された箇所が実際には当該の問題に該当しないもの．非暗号用途でのハッシュ関数の利用や，変数名に基づくヒューリスティックの誤発火が該当する．

悪用可能な欠陥　攻撃者が実際に悪用しうる欠陥．外部から到達可能であり，かつ攻撃者の利益になる結果を生じさせうるもの．

© 2026 Information Processing Society of Japan

表 A·1　要件フレームワークの対象範囲と実証状況
Table A·1　Scope of the Requirement Framework and Empirical Support.

#	TTP カテゴリ	要件	本研究の対応	証拠
1	Supply Chain & Dependencies	P	Trivy による SCA．SBOM をアテステーションの subject に指定．Drift from Upstream は tools_hash 照合	Exp 2 / **Exp 5** (CVE-2026-59950 で適格性喪失，改竄を検出)
2	Command & Code Injection	P	Semgrep SAST．ERROR/HIGH 以上で遮断	Exp 2 (対象コード由来 0–59 件，3/10 通過) / Exp 4
3	Data Exfiltration & Credential Theft	P	p/secrets および外向き通信の自作ルール	**Exp 4** (AWS キーや隠れた POST を検出．agent-scan は検出不可)
4	Protocol Vulnerabilities	P	Missing Integrity Controls に対し署名付きアテステーションを付与	**Exp 3** (改竄ワークフローの署名を拒否)
5	Monitoring & Operational Security	P	不変のビルドログと Rekor 透明性ログが監査証跡を構成	Exp 2/3 (実 Fulcio 証明書・実 Rekor エントリ)
6	Authentication & Authorization	I	Identity Subversion に OIDC + ドメイン所有権．Audit Bypass に job_workflow_ref で固定．**抑止であり防止ではない**	Exp 1 (I-2 = 83.0%) / **Exp 3**
7	Tool Poisoning & Metadata	P/T	実装側の悪意は検出可．記述文側は**専用ルールを要する**．登録後の Tool Mutation は tools_hash 照合	**Exp 4** (実装側を実測) / **Exp 5** (改竄を検出)
8	Privilege & Access Control	T	ケイパビリティ宣言により過剰権限を可視化．宣言の義務化と却下はレジストリの権限内だが，**強制はクライアント依存**	Exp 1 (T-2 = 24.7%) / 宣言の改竄耐性は Exp 5．**網羅性検査は未実証**
9	**Prompt Injection 等（4 カテゴリ）**	—	**対象外**．実行時防御の領分．RADE は宣言により可視化するが，防止はしない	**Exp 4** (観測境界を実測) / Guo ら 成功率 70–100%

表 A·2　Exp 1 の採点項目
Table A·2　Scoring Rubric Used in Exp 1.

項目	判定基準
I-1	公開 URL のドメイン所有者と GitHub リポジトリ所有者の一致
I-2	レジストリ上の名前空間の所有者とリポジトリ所有者の一致
T-1	構造化されたケイパビリティマニフェスト（`mcp.json` 等）の公開
T-2	README またはマニフェストにおける権限・データ取扱いの宣言
T-3	SBOM またはロックファイルによる依存関係の公開
P-1	CI/CD パイプラインの存在（`.github/workflows/` 配下の YAML）
P-2	署名付きアテステーションの公開（Sigstore/in-toto 成果物）
P-3	ワークフローへの署名ステップの組込み（cosign/attest の記述）
S-1	認証やセッションを要求する記述の存在
S-2	外部 API を呼び出す実装の存在

悪用経路のない指摘　指摘自体は正しいが，直接の悪用経路が存在しないもの．推奨される設定が未適用である場合が該当する．

判定は単一の評価者が実施しており，複数評価者による一致率は検証していない（本文第 7.4 節）．

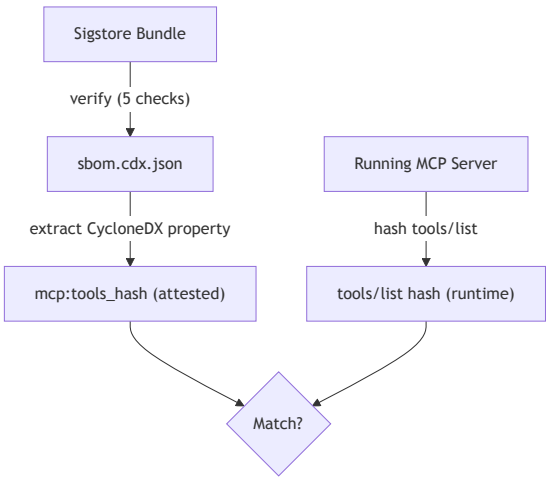


図 A·1　実行時整合性の検証チェーン
Fig. A·1　Verification Chain for Runtime Integrity.

A.5　実行時整合性の検証チェーン

Exp 5 で構築した検証チェーンを**図 A·1** に示す．署名の検証から SBOM，ツールの正規化ハッシュを経て，実行中のサーバが返す `tools/list` のハッシュと照合する．

© 2026 Information Processing Society of Japan

表 A·3　Smithery の特徴量符号化 (真の項目)

Table A·3　Feature Encoding of Smithery (TRUE-valued items).

カテゴリ	特徴量
Monetization	Free
Rights Management	Creator managed DRM
Account Requirement	Some features require registration
Product Type	Extension/add-ons to apps/hardware
Target Audience	General purpose
Type of Product Creators	Business, Community
Intent of App Store	Community building/support, Centralization of product delivery, Expanding a platform popularity/usefulness
Role of Intermediary	CI/CD, Pre-publication checks
Composability	Independent
Analytics	Sentiment and popularity ratings, Product usage data
Communication Channels	Documentation (wikis, FAQs), Product homepage, Promotion/Marketing

A.6　クラスタリング分析の詳細

第 2.4 節の分類は，Zhu ら [17] が公開したアプリストアのデータセットに，MCP レジストリの代表例として Smithery を 1 データ点として追加する形で行った．同データセットは 41 の二値特徴量から構成され，iOS App Store, Steam，DockerHub，npm など多様なプラットフォームを含む．分析手順も Zhu らに従い，二値特徴量の符号化，寄与率 0.05 以上の主成分を保持する PCA，およびシルエット係数により k を選択した k-means 法（$k = 8$）を用いた．Smithery の符号化のうち真となった項目を**表 A·3** に示す．

分類の結果を**表 A·4** に示す．Smithery はコミュニティ主導の非特化型拡張ストアのクラスタに属し，Chrome Web Store および Firefox Add-ons と同一のクラスタとなった．パッケージマネージャのクラスタ（npm，PyPI 等）には属していない．

なお表 A·3 において，Product Type の *Service/Resources* は偽として符号化している．リモート MCP サーバをサービスとみなす立場（第 2.4 節の二重性）を反映してこれを真に変更した場合も，Smithery のクラスタ割り当ては変化しない．すなわち本分析から導かれるガバナンス統制の含意は，この符号化の選択に対して頑健である．

本分析は既存のデータセットへの 1 件の追加であり，統計的な主張としては限定的である．本稿では，移転可能なガバナンス統制を特定するための位置づけの手段として用いており，第 6.1 節の実測（サービス型 92.6%）がこれを補完する．

表 A·4　アプリストアのクラスタ分類

Table A·4　Classification of App Stores into Clusters.

種別	下位分類	クラスタに含まれるストア
拡張機能	商用・特化型	Adobe Magento, Autodesk, BigCommerce, GoG, HubSpot, Plugin Boutique, Presta Shop, SketchUCation, CS-Cart
	コミュニティ・特化型	Bukkit, CurseForge, DockerHub, Home Assistant, JMeter, Kodi, Minion, VSCode Marketplace
	コミュニティ・非特化型	APKPure, Chrome Web Store, Eclipse Marketplace, Firefox Add-ons, GNOME, WordPress, **Smithery**
汎用	商用	AWS, Google Play Store, Microsoft Store, Nintendo eShop, Steam, Samsung Galaxy Store, Apple App Store
	コミュニティ	Chocolatey, F-Droid, Flatpak, Guardian Project Repository, IzzyOnDroid, Snapcraft
パッケージマネージャ		Ajour, Jenkins, MacPorts, npm, NuGet, Packagist, PyPI, Typo3, Ubuntu packages
サブスクリプション型		GitHub Marketplace, JetBrains, Qt Marketplace, concrete5 marketplace
その他		MarketPlaceKit, RICOH THETA, Sellacious, DAZ3D

© 2026 Information Processing Society of Japan